Faculty of Engineering and Physical Sciences
School of Computer Science and Electronic Engineering

MSc programmes in Computer Science

module_id - module_name

Author: Insert Name Here
Date: Submission Date

Table of Contents

1. Originality & AI Use

> **Warning**
>
> - Coursework will be routinely checked for academic misconduct.
> - Copying might constitute plagiarism.
> - TurnItIn will check all submissions for instances of copying and collusion.
> - Your submission must be in your own words. Where you have quoted text from another source this must be clearly indicated and referenced.
> - Complete the statement on AI Use to show how you uses AI to enhance your work.
> - Please read the Student Handbook to ensure you know what this means!

1.1 Statement of originality

I confirm that the submitted work is my own work. No element has been previously submitted for assessment, or where it has, it has been correctly referenced. **I have clearly identified and fully acknowledged all material that should be attributed to others** (whether published or unpublished, and **including any content generated by a deep learning/artificial intelligence tool**, and have also included their source references where relevant) using the referencing system required by my course or in this specific assignment.

I agree that the University may submit my work to means of checking this, such as the plagiarism detection service Turnitin® UK and the Turnitin® Authorship Investigate service. I confirm that I understand that assessed work that has been shown to have been plagiarised will be penalised.

1.2 Statement on AI use

> **Note**
>
> Complete a statement on how you used AI including what you didn't use AI for. Below is an example of the detail required. You should include the tool used, how it was used, the prompts you used and a justification for its use. You should also include a statement on what you didn't use AI for.

Tool Used	Explain How Used	List Prompts	Justify Use
Gemini AI 3.5	Used to generate a list of actors for the system context diagram.	1. List actors for interfaces to an internet store. 2. Add those business and IT actors that support the application, infrastructure and cybersecurity. 3. Add the following list of actors: actor1, actor2, actor3.	Used to identify a comprehensive list of actors and reduce the risk of missing actors.
Gemini AI 3.5	Identify a typical interface for a payment system.	1. What sort of protocol and technology for the interface to the Visa payments system.	Used to identify the payment gateway system actor interfaces to a system.
Gemini AI	Improve explanations in the coursework.	1. Improve the following description to make clearer in no more than two sentences: *<paragraph>*	Used to make the discussion clearer.
Quillbot	Improve overall grammar and writing style as I edit a complex paragraphs.	No prompts - I put into the grammar checker and sometimes use the Improve function.	Used to improve my grammar and writing style as I edit each paragraph.

2. Section

2.1 SubSection

2.1.1 SubSubSection

2.1.2 SubSubSection

2.2 SubSection

2.2.1 SubSubSection

2.3 SubSection

2.3.1 SubSubSection

3. Section

3.1 SubSection

3.1.1 SubSubSection

3.1.2 SubSubSection

3.2 SubSection

3.2.1 SubSubSection

3.3 SubSection

3.3.1 SubSubSection

4. Section

4.1 SubSection

4.1.1 SubSubSection

4.1.2 SubSubSection

4.2 SubSection

4.2.1 SubSubSection

4.3 SubSection

4.3.1 SubSubSection

5. Section

5.1 SubSection

5.1.1 SubSubSection

5.1.2 SubSubSection

5.2 SubSection

5.2.1 SubSubSection

5.3 SubSection

5.3.1 SubSubSection

6. Start here

Documentation is easy to publish on a static website when using a docs-as-code workflow. Markdown is a markup language used for writing documentation in text files that are then stored in a Git repository. Hosted Git services such as GitLab or GitHub, together with tooling such as Visual Studio Code make it easy to maintain and host documentation.

Adopting a docs-as-code workflow transforms documentation from a chore into an engineering process. By treating your written content with the same rigour as code, you enable a collaborative approach to documentation.

6.1 The docs-as-code philosophy

Docs-as-code means using the same tools and workflows for documentation as you do for software development. This creates a unified environment where writers and developers use the same tools and development workflow.

`Markdown as the Source of Truth`

> Markdown is a lightweight markup language that's simple to read in its raw form and consistently renders on the web. It's used for writing documentation for conversion into HTML for web publishing.

`Version control via Git`

> Storing files in a Git repository (such as GitHub, GitLab, or Bitbucket) enables the tracking of all changes to the documentation. There will be a complete history of "who changed what and why," making it easy to undo errors and audit changes.

`Collaborative Reviews`

> Instead of emailing Word docs back and forth, teams use Pull Requests (PRs) or Merge Requests (MRs). This enables peer reviews, automated linting, and transparent discussions before any content goes live.

6.2 The docs-as-code stack

To move from using a word processor or simple website to a scalable and effective documentation tool set needs a stack of tools:

`Authoring tool`

> Effective documentation depends on tools that help with content editing and automate quality checks, including spelling, grammar, style and formatting consistency. While there are numerous text editors available, Visual Studio Code stands out as a favoured option due to its extensive ecosystem of

extensions. By using Visual Studio Code, writers can take advantage of a variety of extensions that offer real-time quality assurance.

`Docs-as-code builder`

Markdown files can serve as a foundation for a static website. However, they often require enhanced formatting options through additional themes and styling. A docs-as-code builder then transforms the Markdown and supplementary instructions into HTML, applying a theme to generate a professional-looking website. Zensical is a fast and reliable docs-as-code builder designed to process Markdown files and create a static documentation website. It enables the website to be viewed locally before publishing, ensuring that the final output meets quality standards.

`Code Repository and Management`

By connecting directly to a Git repository, this integrated environment establishes a secure, centralised vault that tracks the history of project files. Each modification is recorded as a distinct commit, enabling users to audit changes or revert to earlier versions if errors arise. Prior to finalising any work, a pull request prompts a peer-review process that allows collaborators to comment on, test, and approve the updates. This workflow ensures that only vetted, high-quality content is prepared for final publication.

> **Why not LaTeX?**
>
> LaTeX is a typesetting system widely used in academia and for specialised industrial documentation that requires precise formatting. Although it is not built specifically for web publishing, external tools can convert LaTeX source files into HTML for use on static websites. We are using Markdown as it's often preferred for general documentation in industry because it integrates more naturally with modern web-based development workflows.

6.3 Docs-as-code in production

As a student, you will be following a simplified workflow, as you aren't working with documentation that spans thousands of pages and is maintained by a large development team. Nevertheless, understanding the approach used at scale can help you appreciate the value of the skills you will develop. GitLab provides a video that outlines the entire process for their documentation and highlights the importance of the skills acquired through a docs-as-code methodology.

Introduction to using GitLab as a technical writing team Youtube video

🔗 **Watch Video**

6.4 Zensical for docs-as-code

Zensical provides the themes and tools necessary to draft professionaldocumentation in Markdown with instant local previews. Once finalised, you can publish your site by uploading the files to a Git repository. From there, automated pipelines build and deploy the content into a live website.

Zensical, written for speed and reliability using the Rust programming language, publishes documentation in a website.

This documentation template is available for you to fork (clone) into a project that you can use to write your own document or report.

Tip

These instructions are composed in markdown format, which is processed by Zensical. To view the structure of Markdown files, go to the Git repository for the documentation located at the top right of this page. There, you will find markdown files that serve as examples for writing your documentation.

6.5 Documentation structure

The documentation for this document template has been split into a number of sections.

`Install tooling`

: The *Install tooling* section describes how to install the core prerequisite tools and create a fork of the document template. By the end of this section you will have an environment ready to develop your own coursework or documentation. The instructions are available for macOS, Windows 11, and Ubuntu/Debian Linux.

`Start editing`

: The *Start editing* section describes how to edit the documentation and view the changes locally before publishing to a Git repository. It also describes how to synchronise your changes with the Git repository.

`Customise`

The *Customise* section discusses how to configure this template to give it different style and layout to meet your specific needs.

`Markdown basics`

The *Markdown basics* section describes the principles of Markdown and how to use it to write your documentation.

`Zensical basics`

The *Zensical basics* section describes the principles of Zensical and how to use it to create and manage your documentation.

`Additional tooling`

The *Additional tooling* section describes additional tools that can be installed to help you create quality docs-as-code documents. They will take additional effort to install and configure.

`Shell commands`

The *Shell commands* section describes the shell commands used in this documentation template. It's intended as a reference for you to use when writing your own documentation. Students in the past found these commands helpful.

Continue to the next section to get started with the installation of the core tools and creating a fork of this document template.

7. Install tooling

This section takes you through the core installation steps for the tools needed to edit your static website. The instructions are for macOS, Windows 11, and Linux (Ubuntu/Debian). If you are using a different operating system, please refer to the official documentation for that operating system.

> **Tip**
>
> The screenshots below may have small text on your screen. You can click on an image to enlarge it.

The install and configuration starts with the setup of Visual Studio Code.

7.1 Install Visual Studio Code

Visual Studio Code (VS Code) is the selected primary editor for developing your documentation website using Zensical. You can use other editors, but the availability of many plugins in Visual Studio Code will help you edit your documentation more efficiently.

The steps below will help you install VS Code and some essential plugins to edit your documentation. If you have already installed VS Code, check through the steps so you have the plugins installed.

7.1.1 Install Visual Studio Code

1. Start with installing Visual Studio Code. Instructions for macOS,

:material-clock-fast: Install Visual Studio Code

macOS using Homebrew

1. If you use the Homebrew package manager, run this command in your Terminal: `bash brew install --cask visual-studio-code`

Windows 11 using PowerShell

1. Download the VS Code User setup for Windows.
2. Run the installer, `VSCodeUserSetup-{version}.exe`. By default the User setup installs Visual Studio Code to your user profile directory. You can change the install location if you want to install it for all users.

Linux (Ubuntu/Debian) using bash

1. Download the `.deb` package from the official website.
2. Open a terminal and navigate to the directory where you downloaded the `.deb` package.
3. Run the following command to install Visual Studio Code: `bash sudo apt install ./<file>.deb` Replace `<file>` with the name of the downloaded `.deb` file.

Further installation instructions are available on the Visual Studio Code website.

7.1.2 Install Visual Studio Code plugins

VS Code has a rich ecosystem of plugins that can enhance your editing experience. The following plugins are recommended for working with Markdown and Zensical:

1. Install markdownlint plugin for Visual Studio Code from the marketplace. This mardownlint extension checks your markdown files using a library of rules to encourage consistent formatting.
2. Install Even Better TOML plugin for Visual Studio Code from the marketplace. This extension helps manage a TOML file.
3. Install LTeX+–LanguageTool grammar/spell checking plugin for Visual Studio Code from the marketplace to enable spelling and grammar checking for Markdown. Configure the plugin in the settings to use the *language* `en-GB`.

7.2 Install Git with Visual Studio Code

Git is a version control system that enables you to track changes to your code and collaborate with others. You will be using Git to manage your documentation website and push your changes to your **GitLab** or **GitHub** cloud repository.

Next, install the `git` command and configure it for Visual Studio Code. The instructions below are for using with both *GitLab* and *GitHub*.

7.2.1 Install and configure Git

1. As a start, you need to install the `git` command. Follow the instructions below to install or update `git` to the latest stable version.

:material-clock-fast: Install Git

macOS using Homebrew

If you use the Homebrew package manager, run this command in your Terminal to either install or update `git` to the latest stable version:

```
brew update
brew install git
```

Windows 11 using PowerShell

Open up a **PowerShell** window and install `git` using the command, or you can download and installthe official git installer from git-scm.com.

```
winget install Git.Git
```

If you just require an updated version of `git`, you can run the following command in **PowerShell**:

```
winget upgrade Git.Git
```

Linux (Ubuntu/Debian) using bash

Open a terminal and run the following command to install or update `git` to the latest stable version:

```
sudo apt update
sudo apt install git
```

Before connecting to any cloud provider, open your terminal (Terminal on macOS/Debian, Git Bash or PowerShell on Windows 11) and set your global username and email. This is the identity stamped onto your commits. Make sure you use the same email address that you used to register for your GitLab or GitHub account.

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

1. Register for an account on the **GitLab** or **GitHub** cloud instance you will use. If you have already registered, you can skip this step.

> **University of Surrey GitLab**
>
> For the University of Surrey, you will be using the GitLab instance provided by the university at https://gitlab.surrey.ac.uk. When you get to the login page, select the button **Surrey Login**{: .bg-grey} and use your university credentials.

1. Now we need to generate ssh keys to use for authentication with your GitLab or GitHub account. Follow the instructions below to generate a new SSH key pair and add it to your GitLab or GitHub account. It's best practice to use modern, secure `ed25519` keys and create separate ones for GitHub and GitLab.

:material-clock-fast: Generate ssh keys

macOS using Homebrew

1. Open the **Terminal** application.
2. Run the following command to generate a new SSH key pair for GitHub and GitLab. Make sure to replace `your.gitxxx.email@example.com` with your actual email address:

```
ssh-keygen -t ed25519 -C
        "your.github.email@example.com" -f ~/.ssh/
        id_ed25519_github
ssh-keygen -t ed25519 -C
        "your.gitlab.email@example.com" -f ~/.ssh/
        id_ed25519_gitlab
```

3. When prompted, type a strong passphrase.

Windows 11 using PowerShell

1. Open the **PowerShell** application.
2. Run the following command to generate a new SSH key pair for GitHub and GitLab. Make sure to replace `your.gitxxx.email@example.com` with your actual email address:

```
ssh-keygen -t ed25519 -C
        "your.github.email@example.com" -f
        $env:USERPROFILE\.ssh\id_ed25519_github
ssh-keygen -t ed25519 -C
        "your.gitlab.email@example.com" -f
        $env:USERPROFILE\.ssh\id_ed25519_gitlab
```

3. When prompted, type a strong passphrase.

Linux (Ubuntu/Debian) using bash

1. Open the **Terminal** application.
2. Run the following command to generate a new SSH key pair for GitHub and GitLab. Make sure to replace `your.gitxxx.email@example.com` with your actual email address:

```
ssh-keygen -t ed25519 -C
        "your.github.email@example.com" -f ~/.ssh/
        id_ed25519_github
ssh-keygen -t ed25519 -C
        "your.gitlab.email@example.com" -f ~/.ssh/
        id_ed25519_gitlab
```

3. When prompted, type a strong passphrase.

1. Then configure the SSH Config file to use the correct SSH key for each service. Open the SSH config file in your preferred text editor (create it if it doesn't exist) and add the following lines:

   For example using `nano` on macOS or Linux:

   ```
   nano ~/.ssh/config
   ```

   paste the following configuration into the file:

   ```
   # GitLab
   Host gitlab.com
       HostName gitlab.com
       User git
       IdentityFile ~/.ssh/id_ed25519_gitlab


   # GitHub
   Host github.com
       HostName github.com
       User git
       IdentityFile ~/.ssh/id_ed25519_github
   ```

then save and close the file (`Ctrl+O` to save and `Ctrl+X` to exit in nano). On Windows, you can use `Notepad` or any text editor to create the `config` file in the `.ssh` directory.

Make sure to replace the paths with the correct paths to your SSH keys if you used different names or locations.

2. Set the correct permissions for the SSH config file and the private keys to ensure they are secure. If you are using macOS or Linux, run the following commands in your terminal:

```
chmod 600 ~/.ssh/config
chmod 600 ~/.ssh/id_ed25519_github
chmod 600 ~/.ssh/id_ed25519_gitlab
```

Windows handles permissions differently and are normally set to only allow access to the user, but ensure that the private keys aren't accessible to other users.

3. Test the SSH connection to GitHub and GitLab to ensure that the keys are working correctly. Run the following commands in your terminal:

```
ssh -T git@github.com
ssh -T git@gitlab.com
```

If successful, you will see greetings like:

```
Hi username! You've successfully authenticated, but GitHub
does not provide shell access.
Welcome to GitLab, @username!
```

4. You have set a password for the ssh keys and you will be prompted for the password each time you use the key. To avoid this, you can use an SSH agent to cache your passphrase. Follow the instructions below to start the SSH agent and add your keys.

:material-clock-fast: Adding ssh keys

macOS using Homebrew

1. Add your SSH private keys to the already running ssh agent:

```
ssh-add ~/.ssh/id_ed25519_github
ssh-add ~/.ssh/id_ed25519_gitlab
```

Windows 11 using PowerShell

1. Start the SSH agent in the background and set it to start automatically with Windows:

```
Start-Service ssh-agent
Set-Service -Name ssh-agent -StartupType Automatic
```

2. Add your SSH private keys to the agent:

```
ssh-add $env:USERPROFILE\.ssh\id_ed25519_github
```

Linux (Ubuntu/Debian) using bash

1. Add your SSH private keys tto the already running ssh agent:

```
ssh-add ~/.ssh/id_ed25519_github
ssh-add ~/.ssh/id_ed25519_gitlab
```

Essential Practice Moving Forward

When cloning repositories from now on, always use the SSH address, never the HTTPS address.

- **Use:** `git clone git@github.com:username/repo.git`
- **Avoid:** `https://github.com/username/repo.git`

7.2.2 Integrate Visual Studio Code with Git

1. Once the keys are generated and the configuration is complete, now add the SSH keys to the GitHub and GitLab accounts. Follow the instructions below to add your SSH keys to your GitHub and GitLab accounts.

:material-clock-fast: Integrate Visual Studio Code with Git

GitLab

1. Log in to your **GitLab** account in a web browser.
2. In the top-right corner, click on your **profile avatar** and select **Edit profile**.
3. On the left-hand sidebar, select **Access > SSH Keys**.
4. Click **Add new key**{: .bg-blue} and fill out the following details:
    - **Title:** Give it a clear name (e.g., VS Code Extension).
    - **Key:** Paste the contents of your public SSH key file (e.g., `~/.ssh/id_ed25519_gitlab.pub`).
5. Click **Add key**{: .bg-blue} to save the key.

GitHub

1. Log in to your **GitHub** account in a web browser.
2. In the top-right corner, click on your **profile avatar** and select **Settings**.
3. On the left-hand sidebar, select **SSH and GPG keys**.
4. Click **New SSH key**{: .bg-green} and fill out the following details:
    - **Title:** Give it a clear name (e.g., VS Code Extension).
    - **Key:** Paste the contents of your public SSH key file (e.g., `~/.ssh/id_ed25519_github.pub`).
5. Click **Add SSH key**{: .bg-green} to save the key.

7.3 Fork and cloning the doc-template

Forking the documentation template creates a copy of the template into your own GitLab or GitHub cloud account. You will then be able to edit the template locally in Visual Studio Code and publish your own documentation website.

Cloning the documentation template creates a local copy of the template on your computer. You will then be able to edit the template locally in Visual Studio Code and publish your own documentation website.

Table 7.3-1: Fork and Clone Comparison at a Glance

Feature	Fork	Clone
Where's the copy made?	On the remote host (GitHub / GitLab)	On your local computer
Is it a Git command?	No (It's a web platform feature)	Yes (`git clone <url>`)
Who owns the target?	You (it's copied to your account)	You (it's on your machine)
Can you push to it?	Yes	Yes (if you have write access to the remote source)
Primary purpose	To propose changes to a project you don't own	To actually do development work, write code, and make commits

The features of forking and cloning are complementary. You can fork a repository to create your own copy on the remote host, and then clone that fork to your local machine to work on it. The standard workflow is:

1. **Fork:** You find a project on GitHub. You click the Fork button on the website. Now, you have a copy at `github.com/your-username/project`.
2. **Clone:** You run `git clone https://github.com/your-username/project.git` in your terminal. Now, the code is on your laptop.
3. **Work:** You write code, make local commits, and test your changes.
4. **Push:** You run `git push origin main` to send your local changes back up to your cloud fork.
5. **Pull Request:** You go back to the GitHub website and open a Pull Request, asking the original project owner to pull the changes from your fork into their original repository.

> **Note**
>
> In this case, you will be creating your own documentation website, so you won't be submitting a pull request to the original repository. You will be working on your own forked copy of the documentation template.

7.3.1 Fork the doc-template

You may already have a GitLab or GitHub repository containing a Zensical template provided for you. If you do, you can skip this section and go to the next section to clone the repository locally.

1. Start with opening up a browser and go to the doc-template repository on the University of Surrey GitLab.

2. Next, fork the documentation template to create a copy of the template in your own Git cloud account. Follow the instructions below to fork the repository.

:material-clock-fast: Fork the documentation template

GitLab

1. Click on the link to fork a copy of the documentation template to create a copy of the template for your use.

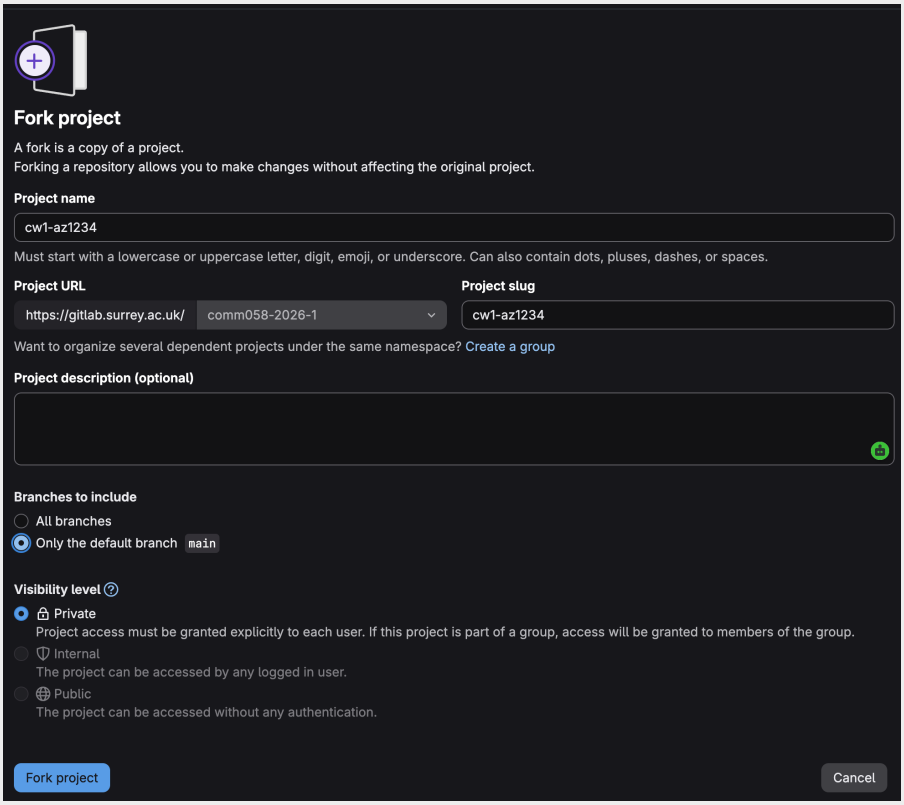


Figure 7.3.1-1: GitLab fork project

2. Enter your *Project name* using to the format the coursework specifies. For example, for coursework 1 for the module COMM058 in the year 2026 for your GitLab ID az1234, enter ‘cw1-az1234’ in the project name field. Edit the *Project slug* to match the project name. Use all lowercase and a dash between words with no spaces.

3. Then select the project namespace for the module as directed by your course tutor.

4. Change the *Visibility Level* to *Private*.

5. Press the button **Fork Project**{: .bg-blue} to create your own copy of the project in the group namespace.

GitHub

xxx

> **Warning**
>
> Don't forget to set the visibility to private, otherwise others can see your repository. Ask someone else to check whether they can see your repository.

7.3.2 Clone the doc-template

This section takes you through the steps to clone the documentation template into your own local device. You will then be able to edit the template locally in Visual Studio Code and eventually publish your own documentation website.

1. Start with creating a directory for all your *GitLab* or *GitHub* projects on your local desktop. For example, create a directory called 'GitLab' in your OneDrive directory..

> **Tip**
>
> Using OneDrive will give you an additional backup of your GitLab repository.

1. Open a terminal (Terminal on macOS/Debian, Git Bash or PowerShell on Windows 11) and navigate to the directory you created in the previous step.
2. Then run the following command to clone the documentation template into your local directory. Replace the `git_website` with the actual GitLab or GitHub website address. For example, `gitlab.com` or `github.organisation.com`. Make sure to replace the `username` with your actual GitLab or GitHub username.

```
git clone git@git_website:username/doc-template.git
```

> **Tip**
>
> You can find your `username` by logging into your GitLab or GitHub account and clicking on your profile picture at the top right corner of the page. On **GitLab** your username will is **below** your name in the dropdown menu. On **GitHub** your username will is **above** your name in the dropdown menu.

7.4 Install Python and Zensical

First we need to install Python. I've written brief instructions below for macOS, Windows 11, and Linux (Ubuntu/Debian). You are also recommended to refer to the official Python installation documentation for your operating system.

If you already have Python installed, you can check the version by running the following command in your terminal or command prompt:

```
python --version
```

The instructions below are for installing Python 3.8 or later. If you have an older version, please update to Python 3.8 or later.

:material-clock-fast: Install Python and Zensical

macOS - Terminal

1. Install python using the Homebrew package manager. If you don't have Homebrew installed, you can install it by following the instructions on the Homebrew website.

   ```
   brew install python
   ```

2. Open *Terminal* and run the following commands to create a virtual environment and install Zensical:

   ```
   # 1. Create the virtual environment
   python3 -m venv .venv

   # 2. Activate it
   source .venv/bin/activate

   # 3. Install Zensical
   pip install zensical
   ```

Windows 11 - PowerShell

1. Download and run the official python installer from the python.org.
2. Open *PowerShell* as an Administrator in your project folder and run:

> **Critical**

````
    Make sure to check the box that says "Add Python to
PATH" during the installation process. This will allow
you to run Python from the command line.

```PowerShell
# 1. Create the virtual environment
python -m venv .venv

# 2. Activate it (Choose the line matching your terminal)
.\.venv\Scripts\Activate.ps1     # ⟵ Use this if you
are in PowerShell
.\.venv\Scripts\activate.bat     # ⟵ Use this if you
are in classic CMD

# 3. Install Zensical inside the environment
pip install zensical
```
````

Linux - Ubuntu/Debian

1. Download the `.deb` package from the official website.
2. Open a terminal and navigate to the directory where you downloaded the `.deb` package.
3. Run the following command to install Visual Studio Code:

```
# 1. Create the virtual environment
python -m venv .venv

# 2. Activate it (Choose the line matching your
        terminal)
.\.venv\Scripts\Activate.ps1     # ⟵ Use this if
        you are in PowerShell
.\.venv\Scripts\activate.bat     # ⟵ Use this if
        you are in classic CMD

# 3. Install Zensical inside the environment
pip install zensical
```

Replace `<file>` with the name of the downloaded `.deb` file.

Further installation instructions are available on the Visual Studio Code website.

7.4.1 Install Zensical Studio plugin

1. Install Zensical Studio Code Extension plugin for Visual Studio Code from the marketplace. This extension provides a set of tools to help you work with Zensical projects, including commands to build and preview your site.

   Follow the instructions on the Zensical Studio plugin page to configure the extension. Add to the `.vscode/settings.json` file in your project directory the following lines:

```
{
  "files.associations": {
    "*.md": "python-markdown"
  }
}
```

There are many other extensions available for Visual Studio Code that can help you with your documentation. You can explore the Visual Studio Code Marketplace to find more extensions that suit your needs.

8. Start editing

8.1 Viewing documentation locally

The great feature of Zensical is that you view the changes to the website locally using a locally hosted website without sending the source to GitLab,

1. Start with installing python and any other software extensions needed. Use the instructions for pip here or uv then here.
2. With cloning (forking) the site, the configuration work to create and publish your site to GitLab is already done for you.
3. You can now go to your top-level working directory in your chosen Python environment (virtual environment or uv) and use the preview function documented here.

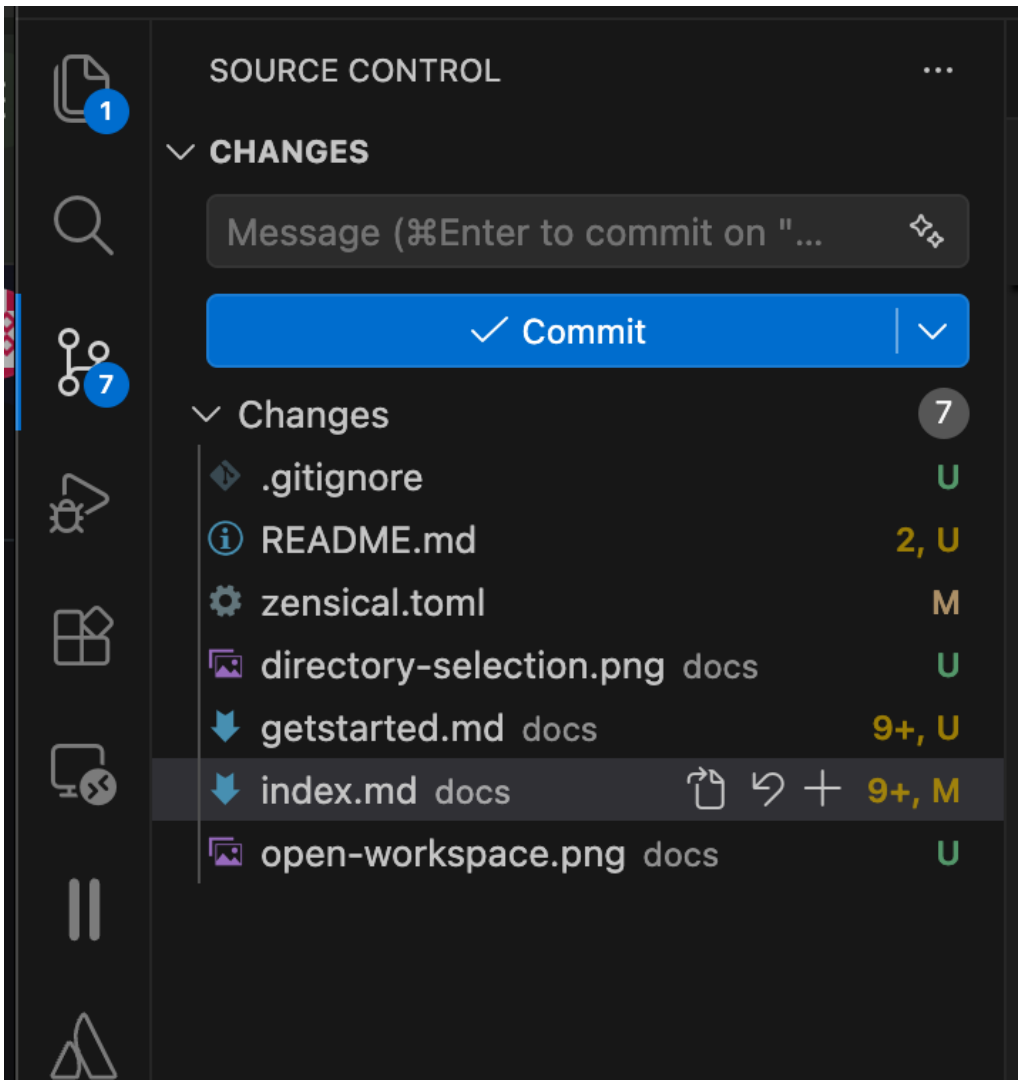
8.2 Perform initial configuration

1. The zensical.toml file contains the configuration for your website.

2. Make sure you save all your files. A file that needs saving will have a filled circle beside the file name in the explorer tab. Select the file and press Ctrl-S/ Cmd-S.

8.3 Synchronise your updates

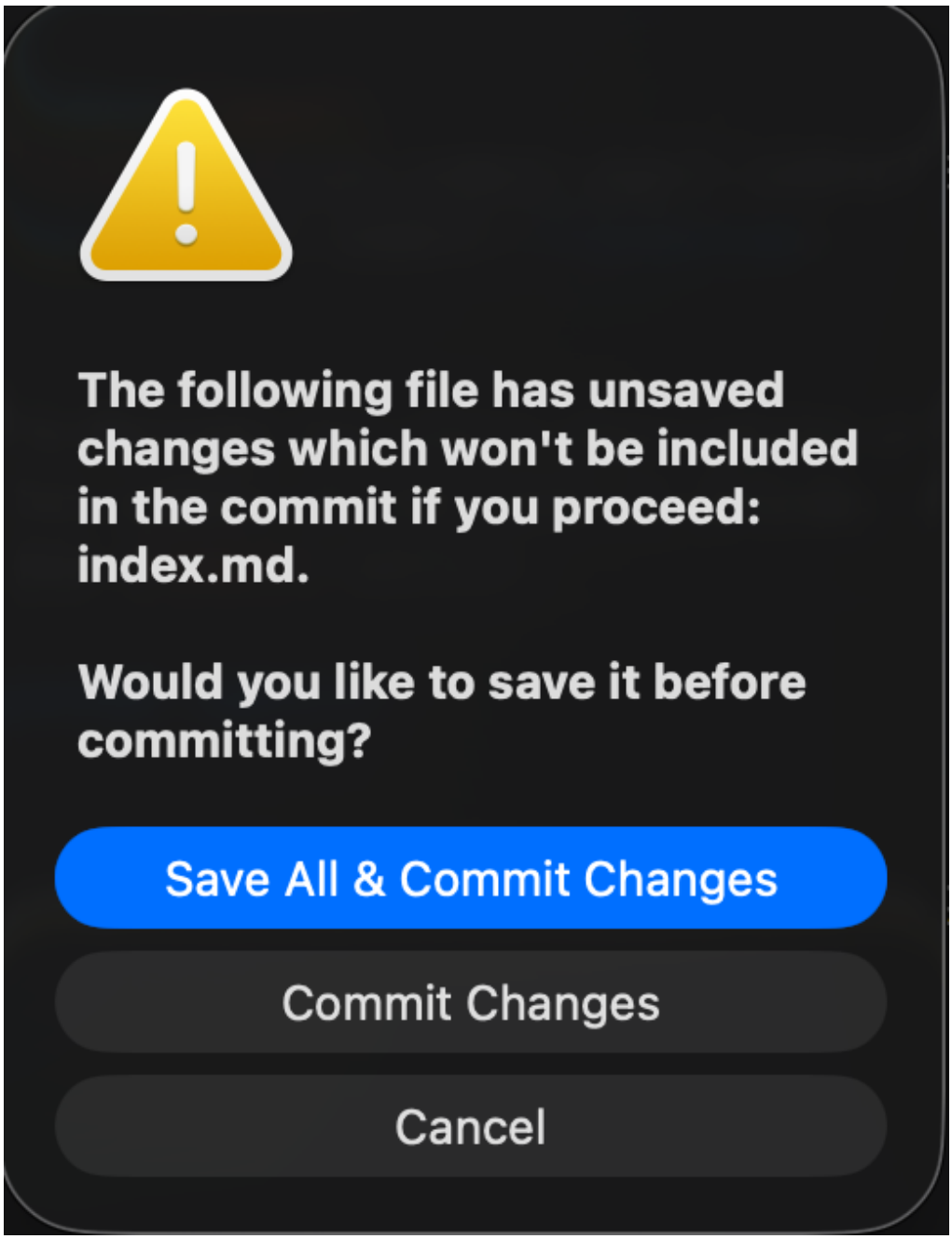
1. Click on the icon on the left in Visual Studio Code and you will see a list of all the changed and mew files.



Initial commit

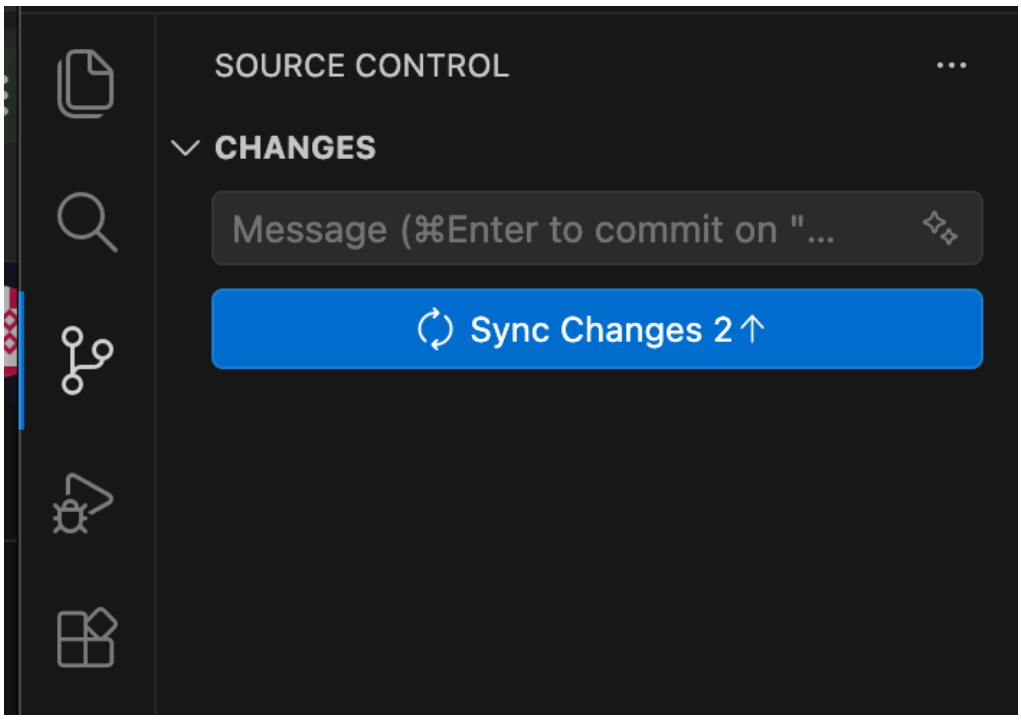
2. Fill into the Message box short description of the change. In this case, enter 'Initial Commit' as this is the first commit of the code to GitLab.

3. Then press the **Commit**{: .bg-blue} button and select **Save All and Commit Changes**{: .bg-blue}.



Commit changes

4. Then sync changes with GitLab

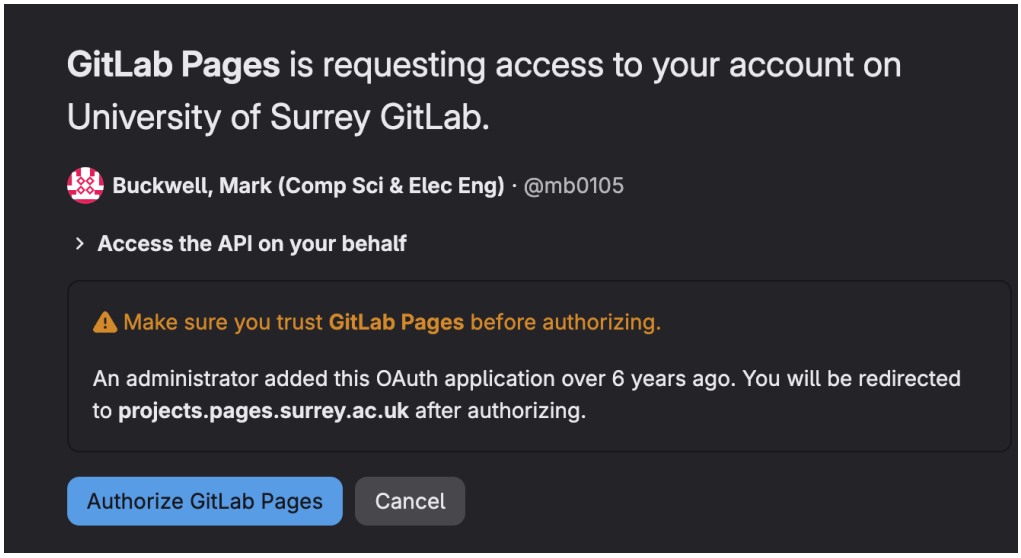


Sync changes

8.4 Viewing online website

1. Enter in the address of your Gitlab Pages website using the form 'https://*namespace*.pages.surrey.ac.uk/*repository-name*'. The address for this template site is http://mb0105.pages.surrey.ac.uk/doc-template

2. A box will pop up for you to authorise access for GitLab Pages to gain access to your project to build it.



Authorise GitLab Pages

3. The browser will redirect you to a site with an additional unique key in the address, such as https://doc-template-4f75ad.pages.surrey.ac.uk/.

8.5 Release your report

Before you release your report, remove the *START HERE* section by commenting it out in the zensical.toml file. Search for START HERE and insert a `#` before the lines between the two comments.

> **University of Surrey Pages Site**
>
> You can still get to the information, as it's on the documentation template website http://mb0105.pages.surrey.ac.uk/doc-template/starthere/starthere.

```
.
nav = [
  {"Cover" = [
    "index.md",
  ]},
  {"Originality" = [
    {"0. Originality & AI Use" = "originality.md"}
  ]},
  {"Assignment" = [
    {"1. Section" = "section1.md"},
    {"2. Section" = "section2.md"},
    {"3. Section" = "section3.md"},
    {"4. Section" = "section4.md"}
  # Comment out the START HERE section (5 lines) before
releasing your report, as it contains instructions for the
author and is not meant for the reader of the report.
  ]},
    {"START HERE" = [
      {"1. Start Here" = "starthere/starthere.md"},
      {"2. Install tooling" = "starthere/installcore.md"},
      {"3. Install extensions" = "starthere/
installextensions.md"},
      {"4. Customise" = "starthere/customise.md"},
      {"5. Markdown basics" = "starthere/markdown.md"},
      {"6. Zensical basics" = "starthere/zensicalbasics.md"},
      {"7. Shell commands" = "starthere/shcommands.md"}
  # Comment until here before releasing your report, as it
contains instructions for the author and is not meant for the
reader of the report.
  ]}
]
```

9. Markdown basics

Markdown is a lightweight markup language that enables you to format plain text using a simple syntax. It's easy to read and easy to write, eventually converting into structurally valid HTML, and is widely used for documentation, readme files, and content management systems. It enables you to focus on writing without worrying about complex formatting, making it an ideal choice for collaborative documentation projects.

It's text-based, meaning you can use a text editor to create and edit Markdown files. This makes it highly portable and compatible with version control systems like Git, enabling collaborative editing and tracking of changes over time. Plain text files with the `.md` extension store the Markdown. You can then share, version, and convert these files into HTML for web publishing.

Below is a summary of the most common formatting elements you'll use in a `.md` file.

> **Zensical Markdown**
>
> Zensical uses a flavour of Markdown called Python Markdown with some extensions. This ensures that your Markdown files are compatible with a wide range of tools and platforms, while also providing additional features for enhanced formatting and functionality. There are some differences between Zensical Markdown and other flavours of Markdown, so it's important to refer to the Zensical documentation for details.

> **Markdown Live Preview**
>
> You can use the Markdown Live Preview website to see how your Markdown will look when rendered. This is a great way to quickly check your formatting and make adjustments as needed.

9.1 Headers

Add hash signs (#) before your text to create a heading. The number of hashes corresponds to the heading level.

```
# H1 Header
## H2 Header
### H3 Header
```

```
#### H4 Header
##### H5 Header
###### H6 Header
```

9.2 Text formatting

You can make text bold, italic, or both to add emphasis without needing complex menus.

```
**bold text**
*italic text*
***bold and italic***
~~strikethrough~~
`inline code`
```

9.3 Links and images

The syntax for these is similar. Just add an exclamation mark at the beginning for an image.

```
[Link text](https://example.com)
[Link with title](https://example.com "Hover title")
![Alt text](image.jpg)
![Image with title](image.jpg "Image title")
```

9.4 Lists

Markdown handles both ordered (numbered) and unordered (bulleted) lists.

9.4.1 Unordered lists

Use a minus sign (-), asterisk (*), or plus sign (+).

```
- Item 1
- Item 2
  - Nested item
```

9.4.2 Ordered lists

Simply use numbers followed by a period.

```
 1. First item
2. Second item
3. Third item
```

9.5 Code blocks

Markdown is a favorite for developers because of how it handles code snippets.

- Inline Code: Wrap text in backticks: `code` .
- Code Blocks: Wrap multiple lines in “fences” using three backticks (```). You can often specify the language for syntax highlighting.

````
 ```javascript
function hello() {
  console.log("Hello, world!");
}
```
````

9.6 Tables

Use pipes `|` and hyphens `-` to create tables. The second row defines the alignment using colons.

```
 | Header 1 | Header 2 | Header 3 |
|----------|----------|----------|
| Row 1    | Data     | Data     |
| Row 2    | Data     | Data     |
```

9.7 Horizontal rule

```
 ***
or
***
or
---
```

9.8 Task lists

```
 - [x] Completed task
- [ ] Incomplete task
- [ ] Another task
```

9.9 Blockquotes and rules

To highlight quotes or separate sections:

- **Blockquote:** Use the `>` symbol before the text to create a callout or quote.

```
 > This is a blockquote
> Multiple lines
>> Nested quote
```

- **Horizontal Rule** Use three dashes `---` on a line by themselves to create a thematic break.

9.10 Quick tips

- **Line Breaks:** To create a line break without a new paragraph, end a line with two or more spaces before hitting enter.
- **Escaping Characters:** If you want to show a literal character (like a *) without it formatting the text, use a backslash: `\*`.

10. Using Zensical

Some brief Zensical notation is below. For full documentation visit zensical.org.

10.1 Commands

Zensical provides a command line interface (CLI) to create, build, and serve your documentation. The following commands are available:

- `zensical new` - Create a new project
- `zensical serve` - Start local web server
- `zensical build` - Build your site

10.2 Examples

Some examples of Zensical syntax are below. For full documentation visit zensical.org.

10.2.1 Lists within lists

Markdown supports nested lists by indenting the inner list by four spaces. This is an implementation-specific feature of Python Markdown used by Zensical, and isn't part of the original Markdown specification.

> **The Four Space Rule**
>
> If you are nesting Tabs, Admonitions, or Code Blocks inside a list, you must indent by exactly 4 spaces. If your ordered list numbering resets to "1", check your indentation! Further background information is on the Zensical authoring section.

10.2.2 Admonitions

Zensical supports admonitions, that highlight blocks of content to draw attention to important information. Admonitions are available for notes, warnings, tips, and more. For further details, go to the admonitions documentation.

> **Note**
>
> This is a **note** admonition. Use it to provide helpful information.

> **Warning**
>
> This is a **warning** admonition. Be careful!

10.2.3 Details

Zensical supports collapsible blocks using the `???` syntax. This is useful for hiding content until the user clicks to expand it. For further details, go to the admonitions collapsible blocks documentation.

??? info “Click to expand for more info”

```
This content is hidden until you click to expand it.
Great for FAQs or long explanations.
```

10.3 Code blocks

Zensical supports fenced code blocks with syntax highlighting. You can specify the language for syntax highlighting by adding the language name after the opening backticks. For further details, go to the code blocks documentation.

``` python hl_lines=“2” title=“Code blocks” def greet(name): printf(“Hello, {name}!”) # (1)!

greet(“Python”)

```
1. Go to [code annotations documentation](https://
zensical.org/docs/authoring/code-blocks/#code-annotations)

    Code annotations enable attaching of notes to lines of
code.

You can also highlight code inline: `#!python print("Hello,
Python!")`.

Content tabs

Zensical supports content tabs, which enables you to present
different content in the same space. This is useful for showing
code examples in multiple programming languages. For further
details, go to the [content tabs documentation](https://
zensical.org/docs/authoring/content-tabs/).
```
```

````
::: {.tabbox title="Python"}

```  python
print("Hello from Python!")
````

:::

Rust

```rust
println!("Hello from Rust!");
```

10.4 Images

Zensical supports Markdown image syntax using the `<figure>` tag to add captions. For further details, go to the images documentation.

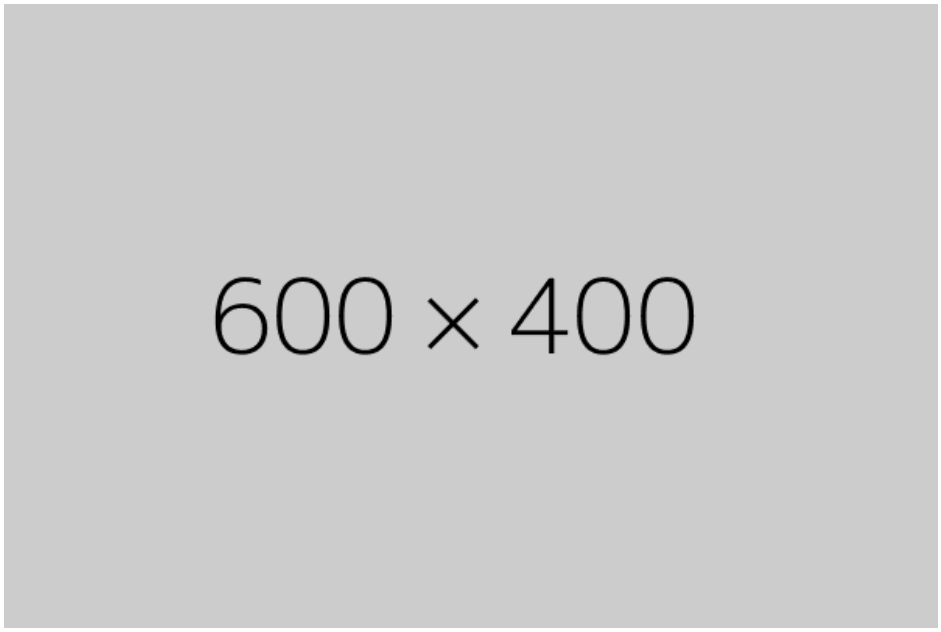


Image title

Image caption

10.5 Diagrams

Zensical supports Mermaid diagrams. You can create flowcharts, sequence diagrams, and more. For further details, go to the diagrams documentation.
```

> **Note**
>
> The COMM058 Architectural Thinking for Security module doesn't use any of these documentation types. It's better that you use draw.io to create your diagrams and export them as images to include in your documentation. Use the downloadable version of draw.io, not the web version, as it's much easier to edit. Also, if you are using draw.io in your working life, your company may have a policy that cloud services can't be used unless it's a paid and approved for hosting confidential company data.

```
graph LR
  A[Start] ⟶ B{Error?};
  B ⟶|Yes| C[Hmm...];
  C ⟶ D[Debug];
  D ⟶ B;
  B ---->|No| E[Yay!];
```

10.6 Footnotes

Zensical supports footnotes, which enables you to add references or additional information without cluttering the main text. You can create a footnote by using the `[^1]` syntax. For further details, go to the footnotes documentation.

Here's a sentence with a footnote.[1]

Hover it, to see a tooltip.

10.7 Formatting

Zensical supports various formatting options, including bold, italics, and strikethrough. You can also create headings, blockquotes, and horizontal rules. For further details, go to the formatting documentation.

- ==This was marked (highlight)==
- ++This was inserted (underline)++
- ~~This was deleted (strikethrough)~~
- H_2O
- A^TA
- ++ctrl+alt+del++

10.8 Icons, emojis

Zensical supports icons and emojis. You can use the `:icon-name:` syntax to add icons from the Lucide icon set, or use standard emoji codes. For further details, go to the icons and emojis documentation.

- :sparkles: `:sparkles:`
- 🚀 `ZICALICONPAD0001`
- :tada: `:tada:`
- :memo: `:memo:`
- :eyes: `:eyes:`

10.9 Maths

Zensical supports mathematical notation using MathJax. You can write inline math using the `$...$` syntax, and display math using the `$$...$$` syntax. For further details, go to the math documentation.

\[\cos x=\sum_{k=0}^{\infty}\frac{(-1)^k}{(2k)!}x^{2k} \]

Needs configuration

Note that MathJax is included via a `script` tag on this page and isn't configured in the generated default configuration to avoid including it in a pages that don't need it. See the documentation for details on how to configure it on all your pages if they're more maths-heavy than these simple starter pages.

10.10 Task lists

Zensical supports task lists, which allow you to create checklists with checkboxes. You can create a task list by using the `- [ ]` syntax for an unchecked item and `- [x]` for a checked item. For further details, go to the task lists documentation.

- [x] Install Zensical
- [x] Configure `zensical.toml`
- [x] Write amazing documentation
- [ ] Deploy anywhere

10.11 Tooltips

Zensical supports tooltips, which allow you to add additional information that appears when the user hovers over a specific element. You can create a tooltip by using the `[text][example]` syntax. For further details, go to the tooltips documentation.

Hover over this text

11. Customising

11.1 Directory structure

As a starting point, the documentation template has the following directory structure:

- 🗀 **docs/** — Holds the documentation source tree.
  - :material-file-document-outline: `index.md` — The cover page of your documentation.
  - :material-file-document-outline: `originality.md` — Your declaration of originality and AI use for you to complete.
  - :material-file-document-outline: `section1.md` — The first section of your documentation for you to edit.
  - :material-file-document-outline: `section2.md` — The second section of your documentation for you to edit.
  - :material-file-document-outline: `section3.md` — The third section of your documentation for you to edit.
  - :material-file-document-outline: `section4.md` — The fourth section of your documentation for you to edit.
  - 🗀 **starthere/** — Contains the “Start Here” section that can be deleted once you are familiar with the template.
    - 🗀 **images/** — Contains images used in the “Start Here” section.
    - :material-file-document-outline: `starthere.md` — Introduction to the “Start Here” section.
    - :material-file-document-outline: `installation.md` — Instructions for installing the required tools.
    - :material-file-document-outline: `customising.md` — Guide for customising the documentation template.
    - :material-file-document-outline: `markdown.md` — Principles of Markdown for writing documentation.
    - :material-file-document-outline: `zensicalbasics.md` — Basics of using Zensical for documentation.
    - :material-file-document-outline: `shcommands.md` — Reference for shell commands used in the documentation.
- 🗀 **src/** — Contains the core application logic.
- :material-file-cog-outline: `zensical.toml` — The project’s configuration file.
- :material-file-document-outline: `.vale.ini` — Configuration file for Vale, a syntax and style checker.
- :material-file-document-outline: `requirements.txt` — Lists the Python dependencies required for the project.
- :material-file-document-outline: `README.md` — The README file for the project, providing an overview and instructions.

- :material-file-document-outline: `LICENSE.md` — The license file for the project in Markdown format, specifying the terms under which the project can be used and distributed. We've used the MIT license for the project used by Zensical. It's a permissive free software license that allows for reuse within proprietary software provided all copies of the licensed software include a copy of the MIT License terms and the copyright notice.
- :material-file-document-outline: `.gitignore` — Specifies files and directories to be ignored by Git version control.
- :material-file-document-outline: `.gitlab-ci.yml` — Configuration file for GitLab CI/CD, defining the pipeline for continuous integration and deployment.
- 🗀 `.gitlab/` — Directory containing GitLab-specific configuration files.
    - :material-file-document-outline: `README.md` — README file for the GitLab configuration, providing details about the CI/CD setup.

11.1.1 Changing the site logo

If the documentation website is part of the university's GitLab service or the location of the website is hosted under the University of Surrey domain, the site logo is automatically changed to the University of Surrey logo. Otherwise, the site logo will use the default logos in the `docs/assets/` directory. You can change the default logo by replacing the existing default logo files with your own logo files named `logo_default_black.png` and `logo_default_white.png`.

> **Warning**
>
> Before releasing your document, comment out the 5 lines of the START HERE section in the zensical.toml file.

12. Additional tooling

12.1 Setup for signing git commits

Files submitted to a Git repository can be signed using a DCO (Developer Certificate of Origin) signing. The benefit of signing your commits is that it provides a way to verify the authenticity of the code and ensure that it has not been tampered with. It also helps to establish trust between contributors and maintainers, as it provides a way to verify the identity of the person who made the changes.

Setting up for DCO (Developer Certificate of Origin) signing in Git is straightforward. Unlike GPG key signing (which uses cryptographic keys to verify your identity), DCO signing is a legal statement asserting that you have the right to submit the code.

DCO signing simply appends a text line at the very bottom of your commit message that looks like this: Signed-off-by: Your Name your.email@example.com

You will have already configured your Git environment with your name and email. Here is how to set up the DCO signing globally, automate it, and fix any commits you forgot to sign.

12.1.1 Signing all Git commits globally

Modern versions of Git allow you to turn on automatic DCO signing globally. This means you can run your usual git commit -m “your message” command, and Git will silently inject the signature for you.

Run this command in your terminal:

```
$ git config --global commit.signoff true
```

12.1.2 Manually signing a single commit

If you want to sign a single commit, you can use the `-s` or `--signoff` option with the `git commit` command. For example:

```
$ git commit -s -m "Your commit message"
```

12.1.3 Setting up VS Code to automatically sign commits

If you are using Visual Studio Code, you can configure it to automatically sign your commits in two ways.

Either add the following lines to your `settings.json` file:

```
 "git.enableCommitSigning": true,
"git.signoff": true
```

or you can you can use the VS Code GUI to enable commit signing. 1. Open the **Settings** panel, search for “commit signing”. 1. Check the box for **Git: Enable Commit Signing** and **Git: Signoff**.

12.2 Count number of words in Markdown files

To count the number of words in Markdown files, you can use the `pandoc` command-line tool. Pandoc is a universal document converter that can convert files from one markup format to another. It can also be used to count the number of words in a Markdown file.

First install `pandoc` if you haven’t already. You can find installation instructions on the Pandoc website, with a short version of the instructions below for macOS, Windows 11, and Linux (Ubuntu/Debian):

:material-clock-fast: Install Visual Studio

macOS using Homebrew

```
 brew install pandoc mactex-fonts-recommended mactex
brew install --cask font-inter font-jetbrains-mono
```

Windows 11 using PowerShell

```
winget install jgm.pandoc
```

For Windows 11, download and install MiKTex from https://miktex.org/download and then run the following commands in a PowerShell session running as an administrator:

```
choco install inter-font jetbrainsmono
```

Linux (Ubuntu/Debian) using bash

```
 apt-get update
apt-get install -y pandoc texlive-xetex texlive-fonts-
        recommended texlive-plain-generic texlive-latex-
        extra
apt-get install -y fonts-inter fonts-jetbrains-mono
```

Check that `pandoc` is installed correctly by running the following command in your terminal:

```
$ pandoc --version
```

This command will display the installed version of Pandoc, confirming that it is installed correctly.

Then you can count the total number of words in a Markdown file by running the following command in your Visual Studio Code terminal:

```
$ find docs -name "*.md" -exec pandoc -f markdown-
        yaml_metadata_block -t plain {} + | wc -w
```

12.3 Installing GitLab or GitHub extensions

VS Code doesn't require any extensions to work with GitLab or GitHub, but installing the relevant extension can make it easier to manage your documentation in GitLab or GitHub. Some features of these extensions include: 1. Viewing issues and merge requests directly in VS Code. 1. Creating and managing issues and merge requests directly in VS Code. 1. Viewing and managing your GitLab or GitHub repositories directly in VS Code. 1. Viewing and managing your GitLab or GitHub CI/CD pipelines directly in VS Code.

> **TO DO**
>
> Provide instructions for installing GitLab or GitHub extensions in VS Code.

12.4 Configuring GitLab or GitHub extensions

1. Now we need to create a personal access token (PAT) with specific permissions for your GitLab or GitHub account. This will allow VS Code to access your GitLab or GitHub account without needing to enter your username and password every time you push changes to the repository.

:material-clock-fast: Create a Personal Access Token (PAT)

GitLab

1. Log in to **GitLab** (either gitlab.com or your organisation's self-managed instance) in a web browser.
2. In the top-right corner, click on your **profile avatar** and select **Edit profile**.
3. On the left-hand sidebar, select **Access > Personal access tokens**.
4. Click **Add new token**{: .bg-blue} (or use the token generation form) and fill out the following details:
    - **Token name:** Give it a clear name (e.g., VS Code Extension).
    - **Expiration date:** (Optional) Set an expiration date according to your team's security policy. You can click on the date and select the last date available.
5. Under **Select scopes**, check the **api** scope and copy the token string.
6. Click **Create token**{: .bg-blue} and copy the token string.

> **Warning**

```
Copy the token string **immediately**. GitLab will only
show it to you once; if you refresh or leave the page, it
is gone forever.
```

GitHub

Review the insstructions for configuring VS Code for Git rep access as it normally uses a browser OAuth workflow. If you are a GitHub Enterprise Server user, the browser flow may not work out of the box. In that case, you can use a personal access token (PAT) instead.

1. Log in to **GitHub** (either github.com or your organisation's self-managed instance).
2. In the top-right corner, click on your **profile avatar** and select **Settings**.
3. On the left-hand sidebar, select **Developer settings > Personal access tokens >Fine-grained tokens**.

> **Note**

```
You may be prompted to reauthenticate with GitHub before proceeding to the next step.
```

1. Click **Generate new token**{: .bg-green} and fill out the following details:
    - **Token name:** Give it a clear name (e.g., VS Code Extension).
    - **Expiration date:** (Optional) Set an expiration date according to your team's security policy. You can click on the date and select the last date available.
2. Select the **Repository access** level for the token. Choose **All repositories** if you want the token to have access to all your repositories, or choose **Only select repositories** and specify the repositories you want to grant access to.
3. Under **Permissions**, select **+ Add permissions** and add the following:
    - **Read and Write** access to **Contents** and **Pull Requests**.
    - **Read-only** access to **Metadata**.
4. Click **Generate token**{: .bg-green} and copy the token string.

> **Warning**

```
Copy the token string **immediately**. GitHub will only show it to you once; if you refresh or leave the page, it is gone forever.
```

1. Next, configure the *GitLab* or *GitHub* plugin to use a personal access token (PAT). The instructions below are for both *GitLab* and *GitHub* plugins.

:material-clock-fast: Configure VS Code for Git repo access

GitLab

GitLab's OAuth flow may not work in some environments, so we will use a personal access token (PAT) instead.

1. Open VS Code and open the **Command Palette**:
    1. For **macOS**, press **Command+Shift+P**.
    2. For **Windows** or **Linux**, press **Control+Shift+P**.
2. Type `Gitlab: Authenticate` and press `Enter`.
3. Choose your GitLab instance
    - Select **GitLab.com** if you use a public cloud instance.
    - Select **Add new instance URL** if you use a self-hosted instance, and type out your full dowmain (e.g., `https://gitlab.yourorganisation.com`).
4. Select **Enter an existing token**
5. Paste in your personal access token (PAT) that you created earlier and hit `Enter`.

The extension will instantly validate the token. If successful, you will see your GitLab status update in the bottom status bar, and your GitLab sidebar panel will populate with your issues and merge requests.

GitHub

The built-in GitHub Authentication provider in VS Code forces a browser OAuth workflow by default. However, if your browser is failing to redirect back to VS Code (due to a firewall, proxy, or strict security settings), you can easily use a token fallback.

1. Click the **Accounts** icon (the profile silhouette) in the bottom-left corner of VS Code.
2. Select **Sign in with GitHub** to use... (or trigger a sign-in via Copilot/ Settings Sync).
3. Click **Allow** when asked to open the external website.
4. Your browser will open GitHub to authorize the app. Click **Authorize Visual Studio Code**.
5. **The Fallback:** If the browser gets stuck or fails to automatically reopen VS Code, GitHub will display a web page saying *"If your browser does not redirect you..."* alongside a **blue box containing an authorization token.** Copy that token.
6. Return to VS Code. Look at the very bottom **Status Bar**; it will say `Signing in to github.com....`
7. Click that status bar text. An input box will open at the top of your editor.
8. Paste the token you copied from the browser page and hit `Enter`.

> **Note for GitHub Enterprise Server Users**

```
If your company hosts its own private GitHub Enterprise
Server, the browser flow won't always work out of the
box. To use a PAT directly, start the sign-in prompt,
click Cancel on the browser authorization popups, and VS
Code will automatically change its prompt to a direct
text field asking you to paste your Enterprise PAT.
```

12.5 Install vale to check for grammar, spelling, and style issues

Vale is a syntax and style checker for writing. You can use it to check your documentation for grammar, spelling, and style issues.

1. To install Vale, follow the instructions below for your operating system.

:material-clock-fast: Install Vale

---

macOS - Homebrew

We use Homebrew to install Visual Studio Code on macOS. If you don't have Homebrew installed, you can install it using the following command in your Terminal:

```
/bin/bash -c "$(curl -fsSL https://
        raw.githubusercontent.com/Homebrew/install/HEAD/
        install.sh)"
```

Once Homebrew is installed, run this command in your Terminal:

```
brew install vale
```

Windows 11 - Winget

Use the Microsoft Windows Package Manager (winget) to install Vale on Windows 11.

Open up a **PowerShell** session running as an administrator and install `vale` using the following command:

```
winget install --id errata-ai.Vale
```

Linux - Ubuntu/Debian

We use snaps to install Vale on Ubunti/Debian Linux. If you don't have snapd installed, you can install it using the following commands:

```
sudo apt update
sudo apt install snapd
```

Once snapd is installed, you can install Vale using the following command:

```
sudo snap install vale
```

You can find further information on installing Vale on Linux on the Vale installation page.

1. Create a .vale.ini file in the top level directory of your project. This file configures Vale and specifies the rules and styles for checking your documentation. You can use the following example `.vale.ini` file as a starting point:

```
StylesPath = styles
MinAlertLevel = suggestion
Packages = Microsoft, Readability, proselint

[*.{md,rst,asciidoc,html}]
BasedOnStyles = Vale, Microsoft, Readability, proselint

Vale.Terms = YES
Vale.Avoid = YES
Vale.Spelling = YES

Microsoft.OxfordComma = YES
Microsoft.Passive = YES
Microsoft.Dashes = YES
Microsoft.Spacing = YES
Microsoft.Wordiness = YES
Microsoft.We = YES
```

2. Create a `styles` directory in the top level directory of your project. Then synchronise the styles specified in the `.vale.ini` file running from the top-level directory of your project:

```
$ vale sync
```

3. Use Vale to check your documentation in Visual Studio Code by installing the Vale VSCode extension and then restarting Visual Studio Code.

   When you open a Markdown file in Visual Studio Code, the Vale extension will automatically check your Markdown files you open for grammar, spelling, and style issues based on the rules and styles. View the results in the *Problems* panel of Visual Studio Code.

   You will find it gwenerates a large numnber of suggestions, some of which aren't relevant to your documentation. You can ignore these and focus on the suggestions that are relevant to your writing style and the requirements of your documentation.

4. One of the most prominent suggestions is to change from passive voice to active voice. This is a good suggestion and recommended in business writing, but it can take time to change all instances of a passive voice to active voice.

   If you need some help for this, here are a few websites that can help you understand how to change from passive voice to active voice:

   1. BBC Bitesize
   2. Communication for Professionals
   3. University of York

   You can also change the `Microsoft.Passive` rule in the `.vale.ini` file to `NO` if you find it too difficult to change all instances of passive voice to active voice.

13. Shell commands

You will be using shell commands if you are operating on Linux or macOS to write your documentation. This section serves as a refresher on the essential commands for navigating and managing your system using either bash or zsh.

> **Tip**
>
> A more detailed reference to the shell commnand line can be found on linuxcommand.org.

13.1 Navigation and basic info

Knowing where you are and how to move is the first step to understanding bash/zsh.

Command	Action
`pwd`	Print Working Directory: Shows exactly where you are.
`ls`	List: Shows files in the current folder.
`ls -la`	List All: Shows hidden files and detailed info (sizes, dates).
`cd [dir]`	Change Directory: Move to a folder (For example, cd Documents).
`cd ..`	Go Up: Moves one level up to the parent folder.
`cd ~`	Home: Takes you back to your user folder.
`clear`	Cleans up the terminal screen (Shortcut: Cmd + K).

13.2 File and folder operations

Next, here are the shell commands to manage directories and files. You will be using these commands to create, copy, move, and delete files and folders.

> **Warning**
>
> A shell doesn't have a "Trash" bin. When you delete something here, it's usually gone for good.

Command	Action
`mkdir [name]`	Make Directory: Creates a new folder.
`touch [file]`	Creates a new empty file.
`cp [src] [dest]`	Copy: Duplicates a file. Use cp -r for folders.
`mv [src] [dest]`	Move: Also used to rename files.
`rm [file]`	Remove: Deletes a file.
`rm -rf [dir]`	Force Remove: Deletes a folder and everything inside (Use with care!).
`cat [file]`	Displays the entire contents of a file in the terminal.
`less [file]`	Opens a file for reading (press q to exit).

13.3 Editing files

You will need to edit files in the terminal. There are many editors available, each with its own strengths and weaknesses. Here are some popular options:

Editor	Style	Best For
`nano`	Lightweight / Modeless	Quick, beginner-friendly configuration edits.
`vim` / `neovim`	Modal / Highly Keyboard-Driven	Rapid editing, coding, and heavy terminal workflows.
`micro`	Modern / Modeless	A modern terminal editor with intuitive Ctrl+C/Ctrl+V shortcuts and mouse support.
`helix`	Modal / Modern	

Editor	Style	Best For
		A modern, fast terminal editor with built-in language server support and multiple cursors.
`emacs`	Extendable Environment	Users who want an entire operating system of utilities inside their editor.

If you are new to terminal editors, start with `nano` or `micro`. If you want to learn a more powerful editor, try `vim` or `helix`.

13.4 Administrative and permissions

Often the permissions of files and folders will need changing. Here are some commands to help you manage permissions and ownership.

> **Note**
>
> If you get a "Permission Denied" error, you likely need sudo.

Command	Action
`sudo [cmd]`	SuperUser Do: Runs a command with admin privileges.
`chmod 755 [file]`	Changes permissions (755 is standard for executable scripts).
`chown [user] [file]`	Changes the owner of a file.
`history`	Shows a list of all recently used commands.

13.5 Viewing content

Handling text files with shell commands.

Command	Action
`cat`	Concatenate: Dumps the whole file to your screen. Best for short files.
`tac`	Reverse cat: Displays the file starting from the last line. Great for seeing the newest entries in a log first.
`nl`	Number Lines: Displays file content with line numbers. Use `nl -ba` to number every single line, including blanks.
`more`	The “classic” version. It lets you scroll down using the Spacebar. Once you reach the end, it exits automatically.
`less`	A more powerful version of ‘more’. It doesn’t load the whole file at once (it’s faster). You can scroll up (using the arrow keys) and you can search within the text (press / then type your word).
`head -n 20 [file]`	Shows you the top 20 lines of a file.
`tail -n 20 [file]`	Shows you the bottom 20 lines.
`tail -f [file]`	This is the “Live” mode where the option keeps the file open and updates the screen in real-time from file additions. This is how developers watch server logs as they happen.

13.6 Searching and filtering

Essential commands for finding files and file contents.

Command	Action
`grep "word" [file]`	Searches for a specific string inside a file.
`find . -name "*.txt"`	Finds all .txt files in the current directory and subdirectories.
`[cmd] | grep "word"`	The Pipe (|) takes the output of one command and sends it to another.

13.7 Essential keyboard shortcuts

There are some essential keyboard shortcuts to help you navigate the terminal more efficiently.

Shortcut	Action
`Tab`	Auto complete. Start typing a folder name and hit Tab. If it's unique, it will finish it for you.
`Arrow Up/Down`	Scroll through your previous commands.
`Ctrl + C`	Cancel/Kill the currently running command.

13.8 Stream shortcuts

Programs and files pass data between themselves using the operators below.

Shortcut	Action
`>`	Overwrite: Sends output to a file, deleting whatever was there before. For example, `echo "hello" > file.txt`
`>>`	Append: Adds output to the end of a file without deleting the old stuff. For example, `echo "world" >> file.txt`
`|`	The Pipe: takes the "Standard Out" of the left command and shoves it into the "Standard In" of the right command.

13.9 Job queue control

There are commands to manage jobs running in the background or suspended in the terminal. It's important to understand that a job is a command that's running in the terminal. You can have multiple jobs running at the same time, and you can control them using these commands.

Shortcut/Command	Action
`[cmd] &`	Start in Background: Runs the command immediately in the background so you can keep typing.
`Ctrl + Z`	Suspend: Freezes the current foreground task and sends it to the background as a “stopped” job.
`Ctrl + C`	Interrupt: Cancel/Kill the currently running command.
`Ctrl + \`	Quit: Force-quits a task and creates a “core dump” (Not often needed for casual use).
`jobs`	Lists all background/suspended jobs.
`fg %[id]`	Foreground: Brings a job back to the front so you can interact with it.
`bg %[id]`	Background: Tells a suspended job to start running again, but in the background.
`kill %[id]`	Terminates a job in your list using its job number.

13.10 Jobs and processes

There are commands available to manage processes and jobs running on your system. A process is a program that’s currently running, and each process has a unique Process ID (PID).

Command	Action
`ps`	“Process Status.” Use ps aux to see every running process.
`top`	A live, updating list of processes (sorted by CPU/Memory). Press q to exit.
`htop`	A prettier, interactive version of top (install via brew install htop).
`pgrep [name]`	Finds the PID of a process by name (For example, pgrep Chrome).

Command	Action
`kill [PID]`	Sends a “polite” request to a process to stop (SIGTERM).
`kill -9 [PID]`	The “Executioner.” Force-kills a process instantly (SIGKILL).
`killall [name]`	Kills all processes with a certain name (For example, killall Finder).

---

1. This is the footnote.[?]